INTERVIEW PREPARATION GUIDE

Playwright Automation Interview Q&A

Experienced QA Automation Engineer · 4 Years' Experience

Practical, first-person sample answers covering TypeScript, Playwright, UI and API automation, framework design, debugging, CI/CD, and real-world testing scenarios.

A preparation resource — personalize examples and results to reflect your actual work.

How to use this guide. These are model answers in an experienced-candidate voice. Keep the structure, but replace project details, tools, metrics, and outcomes with facts you can confidently explain in a follow-up. A strong answer usually gives the context, what you did, why you chose that approach, and the result.

1 · Background and project experience

1. Tell me about yourself.

I'm a QA automation engineer with around four years of experience in functional testing and test automation. I've worked with Playwright and TypeScript to automate browser workflows and APIs, and I'm comfortable contributing across test design, framework maintenance, defect investigation, and CI execution. I focus on building tests that are readable, stable, and useful to the team—not just increasing the test count.

2. Describe your current project and responsibilities.

I work on automating key user journeys for a web application. I review requirements, identify risks and test scenarios, implement UI or API tests, and investigate failures. The framework uses Playwright Test with TypeScript, page objects and shared fixtures for UI tests, a reusable API client for service tests, environment configuration, and reporting. I also review test results and collaborate with developers and other QA engineers to resolve issues.

3. Walk me through your automation framework.

The framework separates test scenarios from reusable implementation. Page objects encapsulate UI operations; shared fixtures provide the page objects to UI tests. API utilities centralize request operations and endpoint paths. Configuration is separated by environment, and hooks handle cross-cutting lifecycle work such as logging and failure screenshots. Playwright Test manages execution, retries, projects, and reports. I try to add abstractions only when they improve reuse or clarity.

4. What is your role in the automation lifecycle?

I participate from planning through maintenance: clarify acceptance criteria, identify testable risks, decide which scenarios belong at UI, API, or lower levels, implement and review tests, run them locally and in CI, and investigate failures. I also update automation when product behavior changes and communicate whether a failure is likely a product defect, test issue, data issue, or environment problem.

2 · Playwright and TypeScript

5. Why did you choose Playwright?

Playwright provides reliable browser automation with auto-waiting, modern locator APIs, isolated browser contexts, multiple browser support, API testing, tracing, and built-in parallel execution. Its Test runner combines assertions, fixtures, retries, and reporting, which reduces the amount of custom infrastructure required. I still evaluate it against the team's application, skills, and CI environment rather than choosing a tool only because it is popular.

6. How does Playwright auto-waiting work?

Before actions such as click, Playwright waits for the locator to resolve to the expected element and for actionability conditions such as visibility, stability, and being enabled. Web-first assertions also retry until the expected state is reached or the timeout expires. I use these built-in waits instead of fixed sleeps, because fixed delays slow down passing tests and still don't guarantee readiness.

7. How do you choose reliable locators?

I prefer locators based on accessible roles and names, labels, or placeholders because they model how users interact with the page. When those are unavailable, I use stable test IDs or scoped selectors. I avoid selectors coupled to styling, generated classes, or fragile DOM depth. I also make the locator specific and check that it uniquely identifies the intended element.

8. What is the difference between a locator and an element handle?

A locator is a query that Playwright can resolve again when an action or assertion runs, so it benefits from auto-waiting and handles changing DOM state more robustly. An element handle refers to a particular DOM element instance and can become stale after the page changes. I use locators for normal test interactions and reserve handles for cases where I specifically need direct access to an element instance.

9. How do you handle waits and dynamic content?

I wait for observable application states: a locator becoming visible, a status message appearing, a URL changing, or a response completing. I use web-first assertions and Playwright's built-in action waits. I avoid arbitrary timeouts and avoid waiting for network idle as a universal solution, since applications with polling or analytics may never truly become idle.

10. Explain browser, context, and page.

A browser is the browser process. A browser context is an isolated session with its own cookies, storage, and permissions. A page is a tab within a context. Playwright Test typically creates an isolated context and page per test, which helps prevent cookies or local storage from leaking between tests and supports safe parallel execution.

11. How do you use fixtures?

Fixtures provide dependencies with a defined setup and teardown lifecycle. I use built-in fixtures such as `page` and extend the test object with reusable page objects where that makes tests simpler. Fixture scope matters: test-scoped resources should be isolated for each test, while worker-scoped resources should be shared only when safe. I avoid placing unrelated setup into a fixture just because it is convenient.

---

12. How do you use hooks responsibly?

I use hooks for common setup, teardown, and cross-cutting concerns, such as logging or attaching diagnostics after a failure. I keep important business steps visible in the test so the scenario remains understandable. In my framework, UI hooks are registered through the shared page-object fixture, which keeps them scoped to those UI tests rather than applying browser work to API-only tests.

---

13. How do you structure Page Objects?

A page object represents a page or meaningful component and exposes clear user-level operations, such as opening a page or submitting a form. It encapsulates locator details and repeated interaction logic. I avoid making it a large repository of every possible locator or putting all test assertions inside it. For shared navigation or common interactions, I use a base page only when the behavior is genuinely shared.

---

14. How do you use TypeScript in test automation?

TypeScript helps catch mistakes early through static types, editor feedback, and safer refactoring. I define types for fixture contracts and structured test data, use clear return types for reusable utilities, and avoid unnecessary `any`. I also keep types practical: they should document the shape of data and improve confidence without turning straightforward test code into a type-system exercise.

---

15. How do you handle dialogs, uploads, and downloads?

For dialogs, I register the dialog handler before performing the action that triggers it, then assert the message or accept/dismiss it as required. For uploads, I use `setInputFiles` with a controlled fixture file. For downloads, I wait for the download event before clicking, then verify the suggested filename or save the file to a known location and validate its contents.

---

16. How do you test in multiple browsers or viewports?

I configure Playwright projects for the browsers and device profiles that matter to the product. I prioritize coverage based on usage and risk rather than running every test everywhere by default. I make sure tests don't depend on browser-specific behavior unless that is exactly what the test is validating, and I use consistent viewport settings for responsive scenarios.

---

3 · API automation and test design

17. How do you decide whether to test at UI or API level?

I test business rules and data combinations at the lowest practical level, often through APIs, because those tests are usually faster and more targeted. I use UI tests for critical user journeys, browser-specific behavior, and integration of visible components. I avoid duplicating every API assertion through the UI; instead I keep a smaller set of valuable end-to-end flows.

18. How do you structure API tests?

I separate endpoint definitions, request-client behavior, test data, and assertions. I validate status codes, response bodies, important headers, and relevant side effects. I include positive, negative, boundary, and authorization cases. I also ensure request contexts are disposed of reliably and avoid coupling tests to mutable data created by other tests.

19. What is the difference between HTTP status and an application response code?

The HTTP status is part of the transport-level response, while an application may also include its own status or error code in the response body. I validate both against the actual API contract. For example, an endpoint may return HTTP 200 while the JSON payload reports `responseCode: 405` for an unsupported method. I wouldn't assume those values must always match without checking the documented behavior.

20. How do you test negative API scenarios?

I cover invalid or missing fields, malformed values, unsupported methods, unauthorized access, not-found resources, and boundary values. I assert the expected status and useful error details, and I check that invalid requests don't create or modify data. I keep each negative test focused on one behavior so a failure points clearly to the broken contract.

21. How do you handle authentication in API tests?

I isolate authentication setup from test assertions. Depending on the application, I may obtain a token through a login endpoint, store it using a token manager, and pass it through a header or cookie. I cover valid, missing, expired, and invalid credentials as appropriate, and I never hard-code real secrets in the repository; CI secrets or environment configuration should supply them.

22. How do you validate response schemas?

For important APIs, I validate that required fields exist, values have the expected types, and key constraints are met. This can be done with explicit assertions or a schema validation library when the schema is reused or complex. I avoid over-specifying fields that are intentionally flexible, because that creates brittle tests whenever harmless response details change.

23. When do you mock API responses?

I mock when I need to isolate the UI from an unstable dependency, exercise an uncommon error state, or make a deterministic frontend test. I keep real integration/API tests as well, because mocked tests cannot prove that the actual service contract works. I label or structure mocked tests clearly so the team understands what is simulated.

24. How do you manage test data and cleanup?

I create independent data per test where possible, using unique identifiers to avoid collisions in parallel runs. I avoid depending on test execution order. If the system supports cleanup, I delete created data in teardown or use an API helper. For shared or production-like environments, I follow data policies and use dedicated test accounts and safe datasets.

4 · Reliability, debugging, and CI/CD

25. How do you diagnose a flaky test?

I look for patterns in the failure: timing, shared data, environment load, locator ambiguity, navigation behavior, or a real product race condition. I inspect traces, screenshots, logs, and retry history, then reproduce with the same browser and conditions. I fix the root cause—such as waiting on the correct state or isolating test data—rather than simply increasing the timeout or accepting retries as normal.

26. What do you do when a test passes locally but fails in CI?

I compare browser versions, environment variables, base URL, test data, viewport, parallel load, and permissions. I use the CI trace and artifacts to locate the point of divergence. I then reproduce as closely as possible, for example by using the same headless mode or worker count. I also check whether the failure is caused by test assumptions such as local state, hard-coded paths, or reliance on execution order.

27. How do you use Playwright trace and screenshots?

A trace provides a timeline of actions, DOM snapshots, network activity, and errors, which is very useful for diagnosing CI-only failures. Screenshots show the visual state at a point in time. I configure artifacts to be available on failures or retries, and I use them to investigate rather than treating them as a substitute for meaningful assertions.

28. How do you use retries?

Retries can help confirm whether a failure is intermittent and preserve diagnostics from a failed attempt. I use them selectively and monitor flaky-test rates. A test that passes only after retry is still a reliability issue worth investigating. Retries should not mask a product defect or become a substitute for deterministic data and synchronization.

29. How do you make tests parallel-safe?

I avoid shared mutable state, use isolated browser contexts, generate unique test data, and make cleanup independent. I don't use one test's output as another test's setup unless there is a deliberate workflow test. For genuinely shared external resources, I isolate them or limit concurrency only for the affected tests instead of serializing the whole suite unnecessarily.

30. How do you integrate tests into Jenkins or CI?

I configure the pipeline to install the correct Node and browser dependencies, load environment values securely, run the selected Playwright suite, and preserve useful artifacts such as HTML/Allure reports, traces, and screenshots. I separate fast smoke checks from broader regression where appropriate. The pipeline should return a clear failure status and make diagnostics accessible to the team.

31. How do you reduce test-suite execution time?

I measure where time is spent first. Then I look for redundant UI coverage that could move to API tests, unnecessary setup, slow fixed waits, and tests that can safely run in parallel. I tune worker count based on available CI capacity and service stability. I preserve meaningful coverage and avoid optimizing solely for runtime if it makes tests less representative.

32. What makes a good automation report?

A good report makes failures actionable: clear test names, expected versus actual results, timestamps, environment and browser details, and links to relevant traces or screenshots. It should distinguish failed, skipped, and flaky tests and be easy for both QA and development teams to use. Reports should support diagnosis, not just display a pass percentage.

5 · Testing fundamentals and scenarios

33. How do you decide what to automate?

I prioritize scenarios that are business-critical, repeated often, stable enough to automate, and expensive or error-prone to test manually. I consider risk, execution frequency, test data needs, and maintenance cost. I don't automate a scenario just to increase coverage numbers; sometimes an exploratory or manual check is more valuable.

34. What is your approach to regression testing?

I identify the changed area, related dependencies, and high-risk user journeys. I run targeted tests first for fast feedback, then the appropriate regression suite. I keep smoke tests small and focused on critical availability, while broader regression checks more business behavior. I review failures and update coverage when the change reveals a missing scenario.

35. How do you report a defect?

I provide a concise title, environment and build details, clear reproduction steps, expected and actual behavior, impact or severity, and supporting evidence such as logs or screenshots. I try to make the issue reproducible and distinguish observed facts from assumptions. I then work with the team to clarify the defect and retest the fix.

36. What do you do when requirements are unclear?

I ask focused questions about expected behavior, edge cases, permissions, and error handling. I use examples or acceptance criteria to make assumptions visible, and I document decisions for the team. If development is blocked, I agree on a reasonable interpretation with the product owner or team rather than silently encoding an assumption into the test.

37. How do you work in Agile?

I participate in refinement and planning to identify testability and risk early, collaborate during development, and provide feedback continuously rather than waiting until the end of a sprint. I keep testing status and blockers visible in stand-ups, review acceptance criteria with product and development, and contribute to retrospectives with practical improvements to quality and delivery.

38. How do you use SQL in testing?

When database access is part of the test process, I use SQL to verify records, investigate test data, and confirm that operations have the expected effect. I use read-only queries when possible and follow access and data-handling rules. I don't make a UI test depend on direct database state unless that is appropriate for the requirement being validated.

39. Tell me about a challenging automation problem you solved.

A strong answer should use a real example and a clear structure: describe the symptom and impact, explain how you investigated it, state the root cause, describe the change you made, and share the result. For example, if a test was intermittently failing, explain what evidence you found in its trace, how you replaced an unreliable wait or isolated its test data, and how you verified the fix across repeated runs. Use your own actual example and metrics.

40. What would you improve in your current framework?

I would start with evidence: flaky-test trends, suite duration, duplicated helpers, and how long it takes to diagnose failures. Based on that, I'd choose a focused improvement such as stronger test-data isolation, clearer API schema checks, or better artifact retention. I'd make the change incrementally, validate it with targeted tests and the full suite, and confirm that it improves maintainability without introducing unnecessary layers.

6 · Short technical follow-ups

41. What is the difference between `expect(locator).toBeVisible()` and `locator.isVisible()`?

The web-first assertion retries until the condition is true or its timeout expires, so it is appropriate when visibility is part of the expected behavior. `isVisible()` returns the current visibility state immediately and does not wait. I use the assertion for test expectations and the boolean method when I genuinely need to branch on the current state.

42. What is the difference between `beforeAll` and `beforeEach`?

`beforeAll` runs once for a test file or describe scope in a worker, so it suits setup that can safely be shared within that scope. `beforeEach` runs before every test and is appropriate for test-specific setup. I'm careful not to put mutable test state into `beforeAll`, because that can cause tests to depend on each other.

43. How do you handle secrets and environment configuration?

I keep credentials out of source control and load them from a secure CI secret store or local environment configuration. I validate required configuration early so a missing value fails with a clear message. I avoid printing secrets in logs or reports and use separate values for different environments.

44. How do you review automation code?

I check that the scenario maps to a clear requirement, assertions verify the intended behavior, locators are stable, data is isolated, and cleanup is safe. I look for unnecessary waits, hidden dependencies, duplicated logic, and sensitive values. I also consider whether the test will produce enough diagnostics to be actionable when it fails in CI.

45. Why should we hire you?

I bring a balanced approach to quality: I can design test scenarios, build and maintain automation, investigate failures, and communicate risk clearly. I value reliable feedback and maintainable code over test counts alone. I'm comfortable collaborating across QA, development, and product, and I'm prepared to learn the team's domain and improve the automation based on its actual needs.

Final interview checklist

- Prepare a 60–90 second introduction that matches your actual experience.
- Have two real examples ready: one framework improvement and one difficult defect or flaky test.

- Be ready to explain one UI test and one API test from your project, including assertions and test data.
- Know your framework's folder structure, execution command, reporting, and CI flow.
- When discussing results, use only metrics you can support with evidence.
- For scenario questions, explain your reasoning and trade-offs, not only the tool or code.

Strong answer formula: Context → action → reason → result → learning. Keep answers direct, use a real example when possible, and be honest about what you personally implemented.